

AES Cache-Timing Laboratory: Trace-Threshold, Cross-Language, and Dial-Sweep Results (supplement to draft 1 — Results and Analysis)

This supplement extends the draft (Introduction + Methodology) with the experiments actually run: the C trace-count threshold, the native cross-language recovery comparison (C vs. Go), the Go garbage-collector ablation, and the dial sweep. Numbering here is local to the supplement; when merged into the main paper the section/figure/table numbers will renumber automatically.

1 Methodology additions

1.1 Dial Sweep at a Fixed Trace Count

The threshold experiment varies trace count while holding the two collection “dials” fixed. A complementary experiment holds trace count fixed and varies the dials, to separate the contribution of *averaging* from that of *signal strength*:

- **-repeat r** : the number of vulnerable encryptions accumulated into one trace. Larger r raises the target operation’s contribution relative to fixed timer overhead (more averaging per trace).
- **-evict-kb e** : the size of the memory region read before each timed operation. Larger e disturbs more of the cache and is expected to strengthen the collision signal.

The count is fixed at **100,000 traces**. This value is chosen deliberately: it sits in the C attack’s *marginal* region, just above its 50% threshold (Section 2.1), where recovery is most sensitive to changes in signal quality. A count deep in the reliable regime (where recovery is already 100%) or deep in the failure regime (0%) would mask any dial effect; the knee maximises the visible dynamic range. We sweep $r \in \{10, 20, 30, 50, 75, 100\}$ and $e \in \{256, 512, 1024, 2048, 4096\}$ KiB, each about the $r = 50, e = 2048$ baseline, with ten interleaved trials per point.

1.2 Native Cross-Language Recovery

The draft’s cross-language plan (Section 2.11) supplies *one shared trace file* to C, Go, and Python to test algorithmic consistency and processing time. The experiments reported here answer a different question. Each implementation performs its *own* measured collection with its native timer and then runs its own recovery, and we compare the resulting *recovery reliability* as a function of trace count. This measures how each language’s runtime affects the quality of the timing channel it can capture—the “separate runtime-dependent experiment” the draft anticipates—rather than the correctness of a shared computation.

Because each language collects independently, the trace data (and its random seed) differ across implementations. Trace count, the fixed AES key, the dial settings, the outlier rule, the recovery

algorithm, and the verification criterion are held identical; only the implementation (and hence its collection runtime) changes. Differences in recovery are therefore attributable to the language’s runtime behaviour during collection, not to the attack logic, which is common. We treat this as a *native-collection* comparison and keep it distinct from the shared-dataset consistency test, which remains future work.

1.3 Garbage-Collector Ablation (Go)

Go executes within a managed runtime whose garbage collector (GC) and goroutine scheduler can preempt execution at unpredictable moments, adding jitter to the per-trace timings the attack depends on. To test whether this runtime noise is the cause of Go’s higher threshold, we repeat the Go collection with two environment settings that quiet the runtime without altering the attack: `GOGC=off` (disables the collector, so no GC pause lands inside a timed region) and `GOMAXPROCS=1` (pins execution to one core, suppressing cross-core migration). Both are read by the Go runtime from the environment; no source change is made. For cost reasons the ablation uses a reduced grid (counts $\{500,000, 250,000, 100,000, 50,000\}$ and a 3×3 dial grid), so it is compared to the garbage-collected run only at matched points and by the fitted 50% threshold, never by a grid-dependent high-reliability count.

1.4 Dataset Governance and Exclusion Criteria

Several exploratory runs preceded the retained runs. To avoid selective reporting, inclusion follows fixed rules, and every excluded run’s metadata is preserved (see the provenance table).

- **Environment gate.** A run is excluded if its mean collection cost exceeds roughly 1.2 ms/sample (baseline ≈ 0.94), a proxy for background load. One 10-trial C sweep is excluded on this basis (≈ 3.0 ms/sample, threefold inflated).
- **Design gate.** Pilot runs with fewer trials, missing reproducibility sidecars, or single-point grids are excluded from the primary analysis and used only for design context.
- **Implementation identity.** One retained C run was launched with an `IMPL=go` tag before the harness supported language selection; the C binary in fact executed. It is verified as C (no `impl` field in its metadata; `min_time` and ms/sample matching C) and relabelled accordingly. This mislabelling is documented because an earlier positional (rather than by-name) parse of the result column produced incorrect tallies; all reported counts are recomputed by column name.

2 Results

2.1 Trace-Count Threshold (C)

Table 1 reports, for the canonical C run, the verified recovery rate at each trace count with 95% Wilson score intervals. Recovery rises from 0% at $\leq 35k$ to a reliable 100% at $\geq 100k$. Fitting a logistic model in $\log_2(\text{count})$ places the 50% threshold at $N_{50} \approx 51,346$ traces ($2^{15.65}$); a model-independent linear interpolation agrees at $\approx 50,000$ ($2^{15.61}$). The attack first reaches high reliability ($\geq 90\%$) at $\approx 75k$ traces (Fig. 1).

Across all counts the mean `min_time` stays within 112–121 ticks and the mean attack score within $\approx 17,200$ – $17,460$ (Fig. 2). Neither trends with count: the per-trace leak is unchanged, and

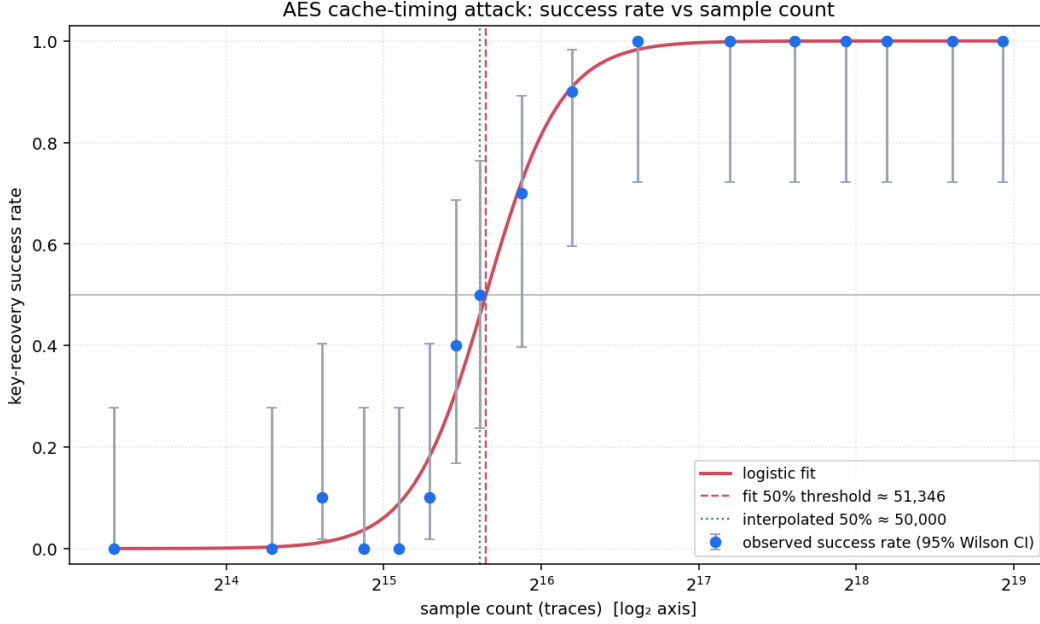


Figure 1: C key-recovery success rate vs. trace count ($\log_2$ axis) with 95% Wilson intervals, the fitted logistic curve, and the 50% threshold ($N_{50} \approx 51,346$).

what improves with more traces is only the stability of the group-mean estimates. The threshold is thus a statistical (averaging) effect, not a signal cliff.

2.2 Cross-Language Recovery

Under identical counts, key, dials, and recovery logic, the three implementations reach 50% recovery at very different trace counts (Table 2, Fig. 3). Go requires $N_{50} \approx 265,283$ ($2^{18.0}$) with the garbage collector enabled—about $5.2\times$ the C threshold. Python is deferred (native collection at these counts is prohibitively slow and expected to recover near 0; see limitations).

2.3 Runtime-Noise Ablation (Go GC on vs. off)

Disabling the Go garbage collector and pinning to one core lowers the fitted threshold only marginally, from $N_{50} \approx 265\text{k}$ to $\approx 243\text{k}$ ($\approx 8\%$), well inside the sampling uncertainty. At matched counts the GC-off run is equal or slightly better everywhere (Table 3), but the $\approx 5\times$ gap to C is essentially untouched. Quieting the runtime helps a little; it does not explain Go’s disadvantage.

2.4 Dial Sweep

At the fixed 100k count, the C dial sweep (Table 4) shows that increasing `-repeat` beyond the $r = 50$ baseline does *not* improve recovery and appears to reduce it (70% at $r = 50$ vs. 20% at $r = 75, 100$), while the strongest cache disturbance, $e = 4096$ KiB, gives the highest recovery (100%). The individual points carry wide intervals ($n = 10$); the robust reading is that signal strength (`-evict-kb`) matters more than additional averaging (`-repeat`) at this count (Fig. 4).

The Go dial sweep is uninformative by construction: 100k traces lie far below Go’s $\approx 265\text{k}$ threshold, so recovery is ≈ 0 (1/110 with GC on, 1/60 with GC off) regardless of dial setting. This

Table 1: C recovery rate vs. trace count (canonical run; 10 trials each, 95% Wilson CI). $N_{50} \approx 51,346$.

Traces	Recovered	Rate	95% CI
$\geq 100,000$	10/10	100%	[72, 100]
75,000	9/10	90%	[60, 98]
60,000	7/10	70%	[40, 89]
50,000	5/10	50%	[24, 76]
45,000	4/10	40%	[17, 69]
40,000	1/10	10%	[2, 40]
$\leq 35,000$	0–1/10	$\leq 10\%$	[0, 40]

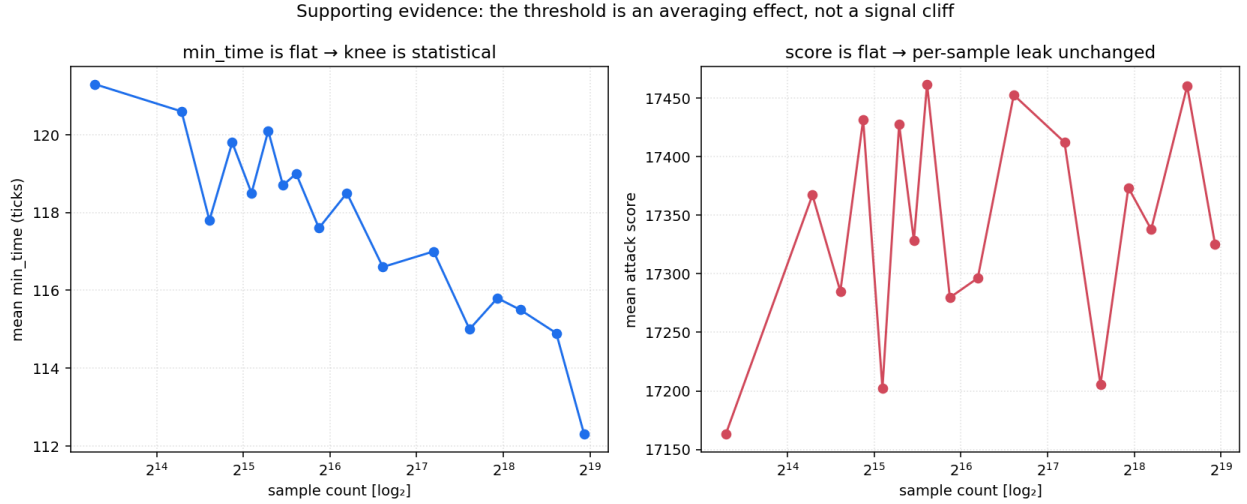


Figure 2: Supporting evidence: mean `min_time` (left) and mean attack score (right) are flat across trace count, so the per-trace leak is unchanged and the threshold reflects averaging quality rather than a signal cliff.

is a consequence of fixing the dial count at C’s knee, which is below Go’s operating point—a design observation, not a null effect (Section 3.3).

3 Analysis

3.1 The Threshold Is an Averaging Effect

The flatness of `min_time` and score across trace count (Section 2.1) localises the mechanism. The per-trace cache-collision signal—the small timing difference the attack exploits—is a fixed quantity that does not depend on how many traces are collected. What changes with count is the variance of each group mean $T_{i,j}[d]$: with more traces per ciphertext-difference group, the correct final-round relation separates from the noise floor more reliably. Recovery is therefore governed by whether the group-mean estimates are stable enough to rank the correct relation first, which is a smooth, probabilistic function of count—hence the psychometric curve rather than a hard boundary.

Table 2: Fitted 50% trace threshold by implementation (identical parameters).

Implementation	N_{50} (traces)	$\log_2 N_{50}$	Ratio to C
C	51,346	15.65	1.0×
Go (GC on)	265,283	18.02	5.2×
Go (GC off)	243,335	17.89	4.7×

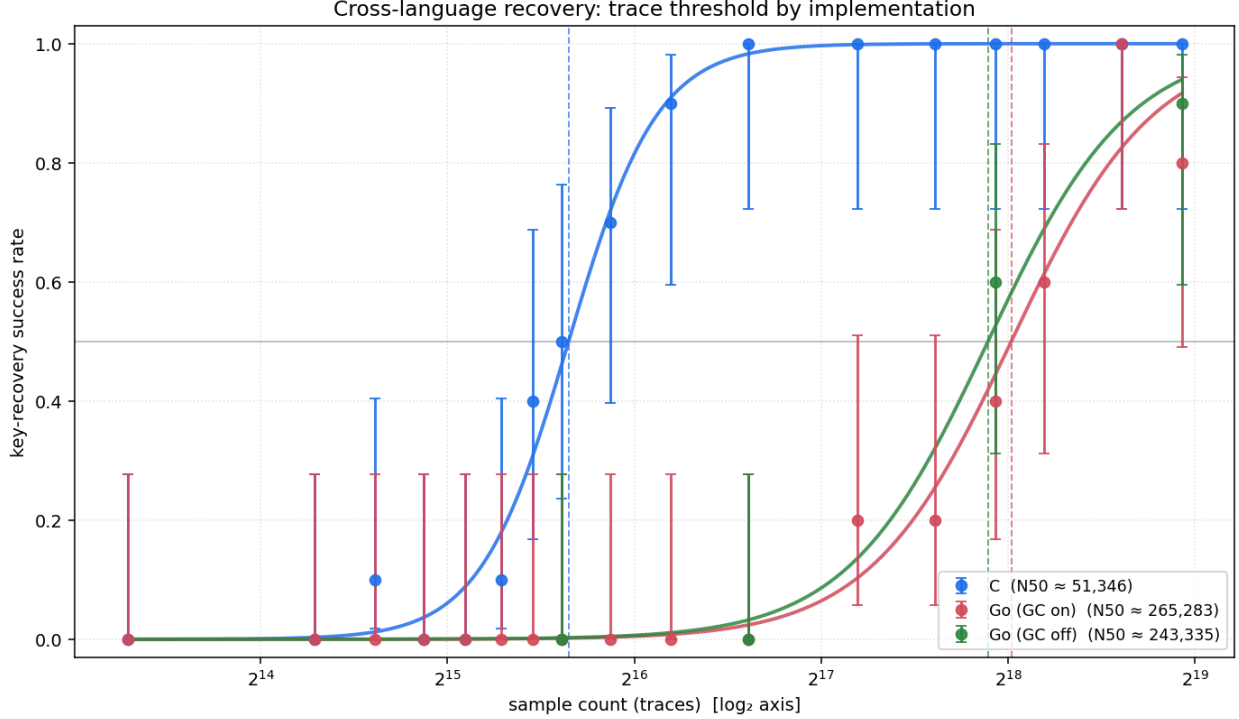


Figure 3: Cross-language recovery. C (blue) reaches 50% recovery near 50k traces; Go with the garbage collector on (red) and off (green) require roughly five times as many, their curves nearly coincident. Points show 95% Wilson intervals; dashed lines mark each fitted N_{50} .

3.2 Why Go Needs About Five Times More Traces

The same attack, on the same key, recovers with $\approx 5\times$ more traces in Go than in C. Because the recovery logic is common, the difference originates in collection: Go’s measured timings are a noisier estimate of the same underlying signal, so more traces are needed to average that noise down to the level where the correct relation wins. The garbage-collector ablation (Section 2.3) sharpens this. If GC pauses were the dominant noise source, disabling the collector would move the threshold substantially; instead it moves N_{50} by only $\approx 8\%$ and leaves the gap to C intact. The residual noise is therefore not the collector but the broader managed-runtime measurement path—bounds-checked array access, scheduler quanta, and coarser or higher-overhead timing than the C collector. In short: Go’s disadvantage is real, reproducible, and a property of *measurement*, not of the algorithm or of the garbage collector specifically.¹

¹ Absolute `min_time` values are far larger in Go than in C; whether that reflects timer units or true overhead requires inspecting each collector’s timing primitive, so we base the cross-language claim on recovery reliability rather than on raw timing magnitudes.

Table 3: Go recovery at matched counts, garbage collector on vs. off (10 trials each).

Traces	GC on	GC off
500,000	8/10	9/10
250,000	4/10	6/10
100,000	0/10	0/10
50,000	0/10	0/10

Table 4: C dial sweep at 100k traces (10 trials each). Left: `-repeat` at $e = 2048$. Right: `-evict-kb` at $r = 50$.

<code>-repeat</code>	Rate	<code>-evict-kb</code>	Rate
10	60%	256	80%
20	50%	512	60%
30	50%	1024	10%
50	70%	2048	50%
75	20%	4096	100%
100	20%		

3.3 Dials: Signal Strength over Averaging, and a Design Lesson

Within C at the 100k knee, additional `-repeat` does not help and the largest `-evict-kb` helps most. This is consistent with the averaging/signal split above: at a count near the threshold, the binding constraint is the strength of the per-trace collision signal (improved by stronger cache eviction), not additional per-trace averaging—and very large `-repeat` may even lengthen each measurement enough to admit more environmental perturbation. The Go dial sweep carries a methodological lesson rather than a result: fixing the dial-sweep count at C’s knee (100k) placed it below Go’s operating threshold ($\approx 265k$), so the sweep could not probe Go’s dial sensitivity. A per-implementation dial count, chosen relative to each implementation’s own threshold, would be required to compare dial effects across languages.

3.4 Positioning Against Prior Work

The implemented attack follows the final-round cache-collision model of Bonneau and Mironov, whose expanded final-round attack recovers a key from a median of 2^{13} – $2^{14.3}$ samples under cache eviction on 2006-era processors. Our C threshold ($N_{50} \approx 2^{15.6}$; high-reliability $\approx 2^{16.2}$) is roughly one order of magnitude higher. This is expected rather than contradictory: the comparison mixes algorithm with platform and instrumentation. Their figures use hardware cycle counters on a Pentium III/UltraSPARC, whereas this laboratory records aggregate wall-clock time on an Apple M4 with a very different cache hierarchy and timing resolution. The agreement to within an order of magnitude, on entirely different hardware and with a coarser timer, indicates that the leakage mechanism reproduces as predicted; the absolute count is a property of this platform and instrument, not a universal constant.

3.5 Threats to Validity

- **Single key.** All runs reuse one fixed AES key to isolate trace count; thresholds may vary across keys.

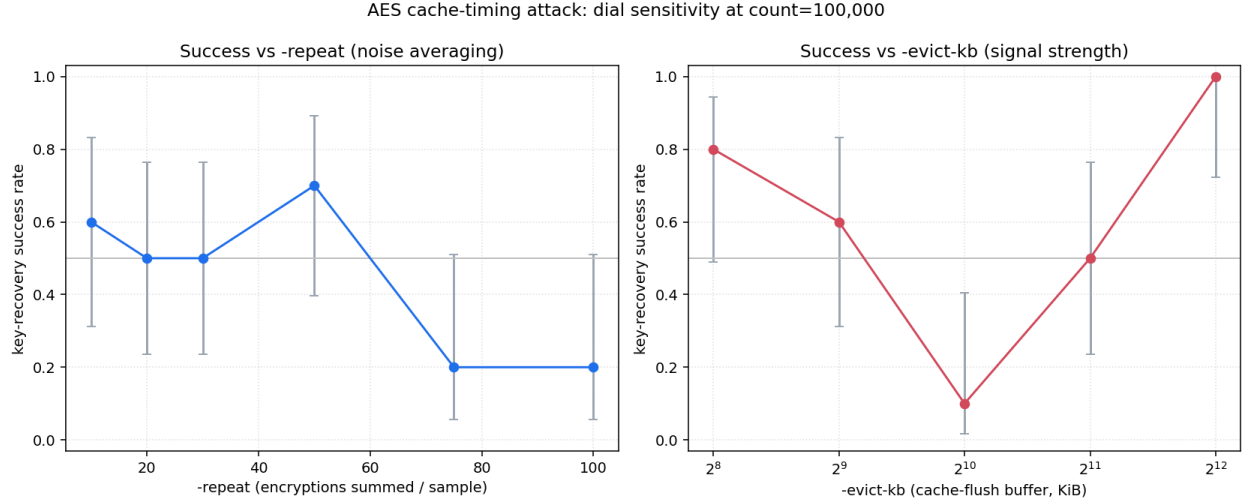


Figure 4: C dial sweep at 100k traces: recovery rate vs. `-repeat` (left) and vs. `-evict-kb` (right), with 95% Wilson intervals. Stronger cache eviction raises recovery; additional repetition does not.

- **Sampling uncertainty.** Ten trials per count yield wide Wilson intervals, especially near 0/10 and 10/10; individual dial points should not be over-interpreted, and N_{50} is an estimate, not a sharp value.
- **Run-to-run environment sensitivity.** Two clean C sweeps place the threshold near 50k and 100k respectively, and a contaminated sweep near 150k; the threshold is sensitive to background load, which we control by the environment gate but cannot eliminate on a general-purpose machine.
- **Native-collection confound.** The cross-language comparison uses each language’s own traces (different seeds), so it measures runtime-in-collection, not shared-dataset algorithmic consistency; the latter (draft RQ2) and stage-level processing time (RQ3) remain future work.
- **Reduced Go(GC-off) grid and platform confound.** The ablation uses a coarse count grid, and the prior-work comparison spans different hardware eras; both are stated where the numbers are used.